

Machine learning 1

Rongyu Chen(A17708249)

2026-04-23

#First up kmeans()

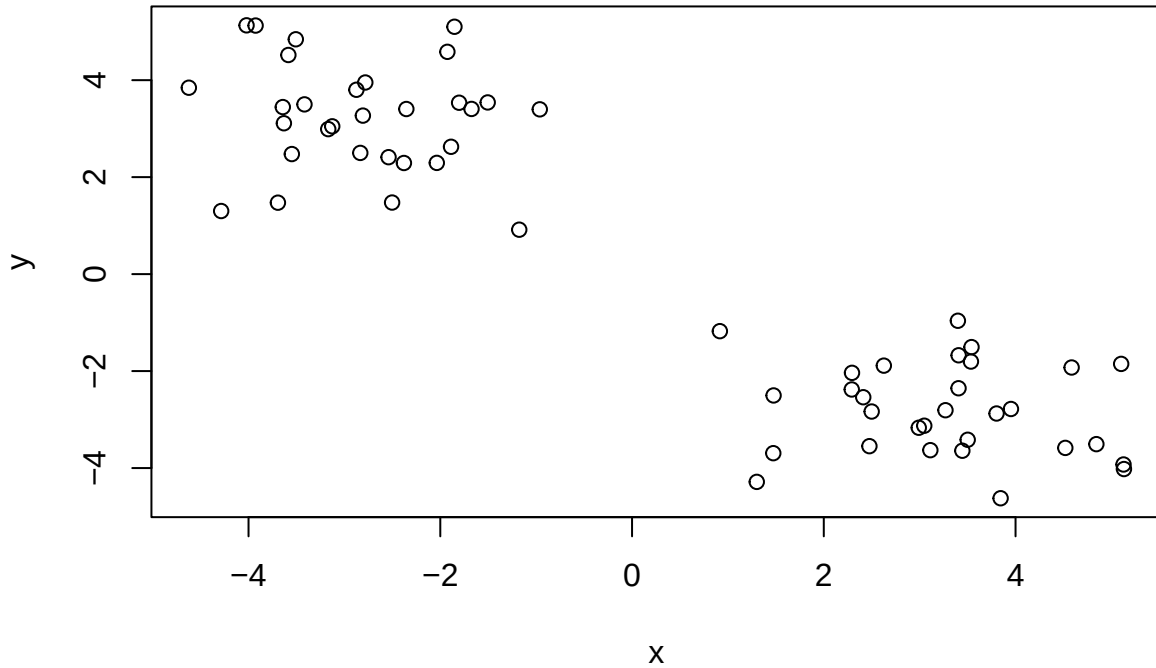
Demo od using kmeans() function in base R. Frist make up some data with a known structure.

```
x <- c(rnorm(30,-3),rnorm(30,3))
x <- cbind(x,y = rev(x))
x
```

```
##           x           y
## [1,] -2.5024625  1.4762362
## [2,] -4.0205437  5.1306392
## [3,] -1.9260159  4.5849240
## [4,] -1.8521632  5.1015220
## [5,] -4.6225938  3.8441222
## [6,] -1.6744029  3.4063616
## [7,] -3.9264559  5.1262031
## [8,] -1.8035462  3.5346719
## [9,] -2.3540447  3.4051980
## [10,] -0.9622328  3.3985398
## [11,] -3.6313126  3.1113739
## [12,] -1.5054324  3.5406436
## [13,] -2.8748803  3.8018973
## [14,] -3.5845925  4.5189688
## [15,] -2.3793257  2.2915856
## [16,] -2.8342875  2.4995354
## [17,] -3.1687180  2.9905087
## [18,] -2.7819603  3.9520961
## [19,] -4.2852642  1.3009942
## [20,] -3.1269559  3.0477641
## [21,] -3.5483850  2.4758626
## [22,] -3.4166285  3.5008958
## [23,] -2.8073192  3.2688793
## [24,] -1.1765572  0.9160722
## [25,] -2.5394318  2.4114302
## [26,] -3.6427414  3.4451643
## [27,] -2.0363549  2.2942195
## [28,] -3.5070395  4.8430544
## [29,] -3.6934033  1.4731144
## [30,] -1.8877834  2.6261886
## [31,]  2.6261886 -1.8877834
## [32,]  1.4731144 -3.6934033
## [33,]  4.8430544 -3.5070395
## [34,]  2.2942195 -2.0363549
## [35,]  3.4451643 -3.6427414
```

```
## [36,] 2.4114302 -2.5394318
## [37,] 0.9160722 -1.1765572
## [38,] 3.2688793 -2.8073192
## [39,] 3.5008958 -3.4166285
## [40,] 2.4758626 -3.5483850
## [41,] 3.0477641 -3.1269559
## [42,] 1.3009942 -4.2852642
## [43,] 3.9520961 -2.7819603
## [44,] 2.9905087 -3.1687180
## [45,] 2.4995354 -2.8342875
## [46,] 2.2915856 -2.3793257
## [47,] 4.5189688 -3.5845925
## [48,] 3.8018973 -2.8748803
## [49,] 3.5406436 -1.5054324
## [50,] 3.1113739 -3.6313126
## [51,] 3.3985398 -0.9622328
## [52,] 3.4051980 -2.3540447
## [53,] 3.5346719 -1.8035462
## [54,] 5.1262031 -3.9264559
## [55,] 3.4063616 -1.6744029
## [56,] 3.8441222 -4.6225938
## [57,] 5.1015220 -1.8521632
## [58,] 4.5849240 -1.9260159
## [59,] 5.1306392 -4.0205437
## [60,] 1.4762362 -2.5024625
```

```
plot(x)
```



Now we have some made up data in 'x'. Let's see how kmeans work with this data.

[illegible]

Q. How many points are in each cluster

```
## [1] 30 30
```

Q. How we do get to the cluster membership/assignment.

k\$cluster

[illegible]

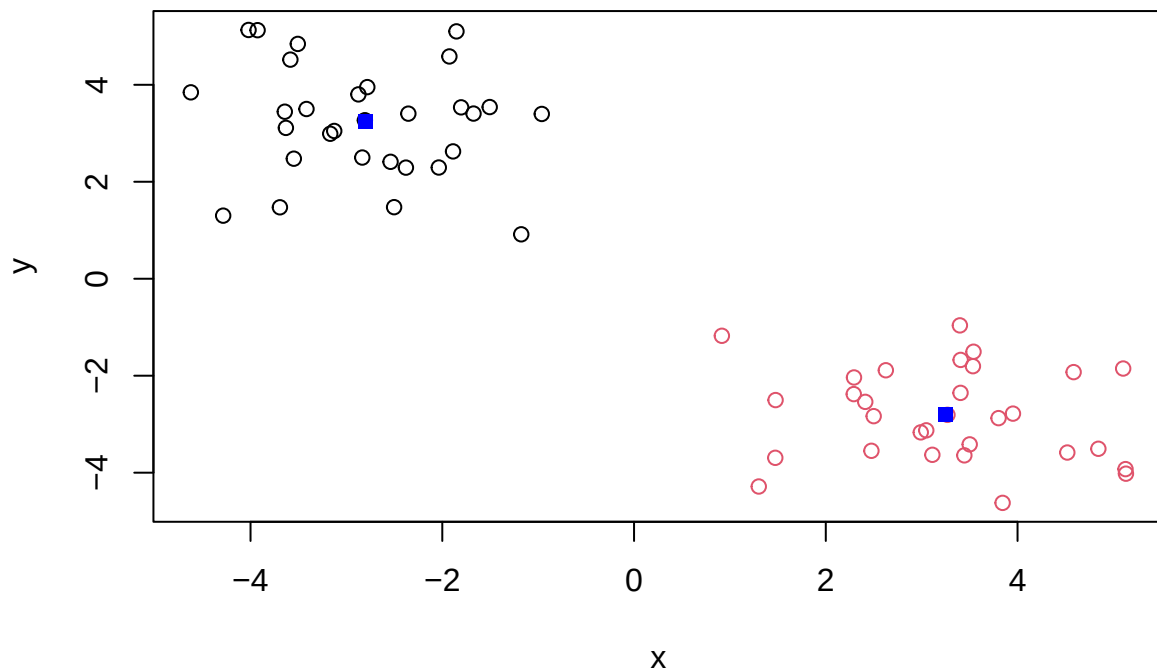
Q. How about cluster center?

k\$centers

```
##          x          y
## 1 -2.802428  3.243956
## 2  3.243956 -2.802428
```

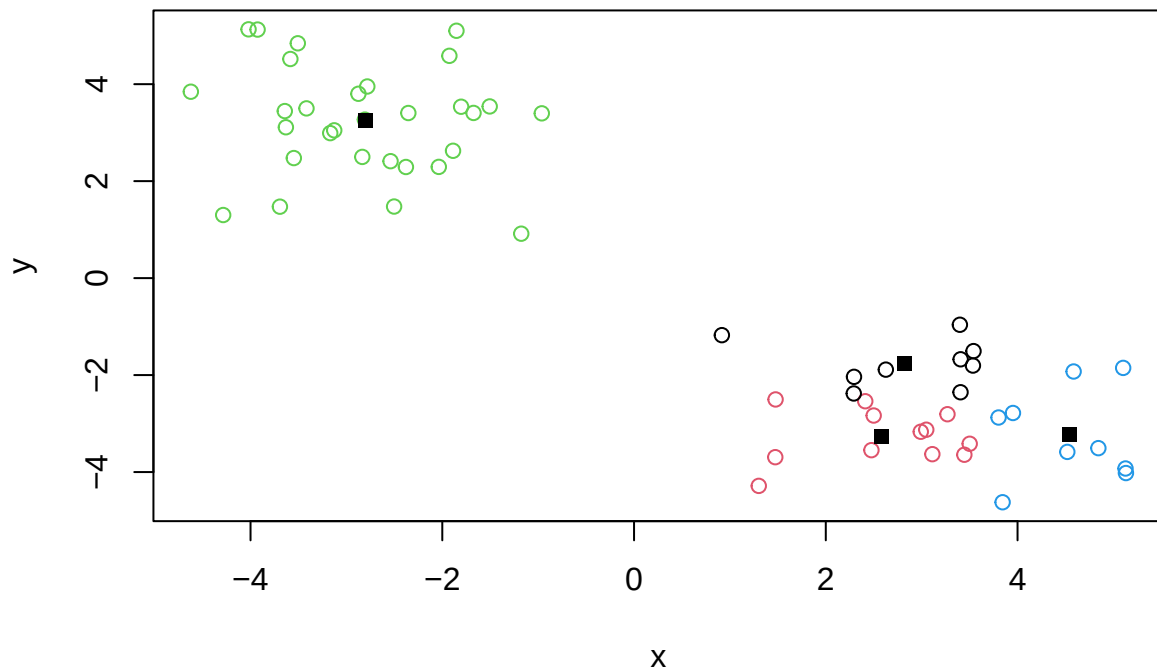
Now we got to the main results. Let's use them to plot data with the kmeans result

```
plot(x,col=k$cluster)
points(k$centers,col="blue",pch=15, cex=1)
```



K-means clustering is widely used because it is fast and relatively simple: it takes numeric data as input and returns a vector indicating cluster membership. However, a key limitation is that we must specify the number of clusters in advance when using `kmeans()`.

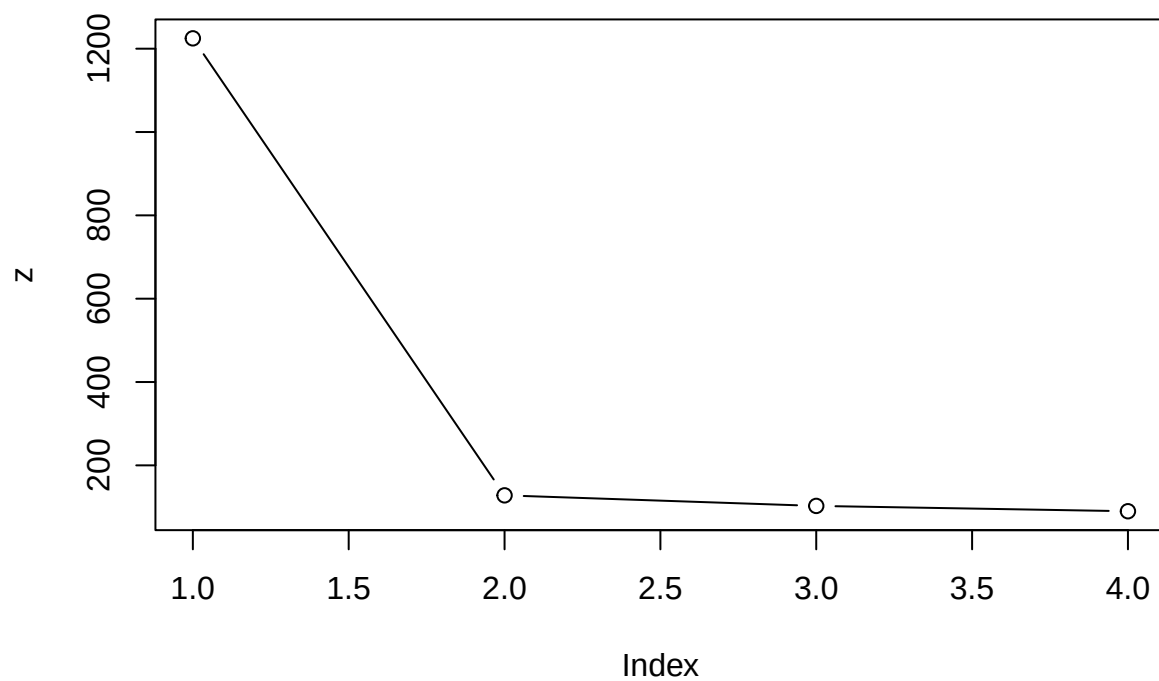
```
k4 <- kmeans(x, centers = 4)
plot(x, col=k4$cluster)
points(k4$centers, pch=15)
```



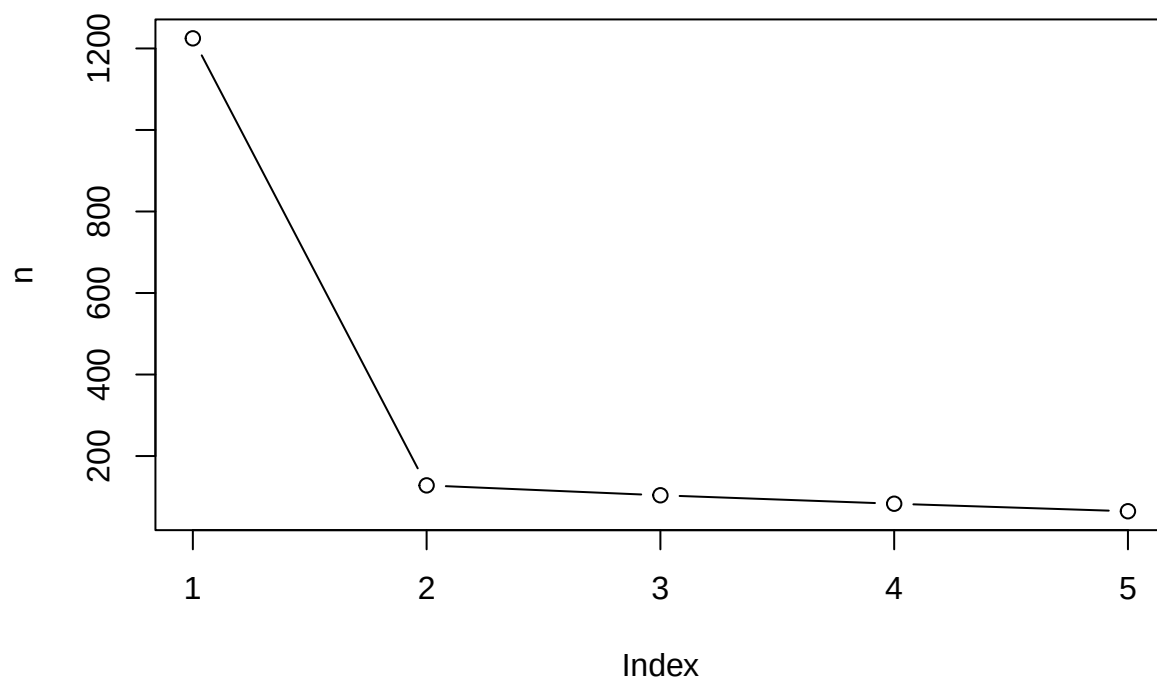
Scree plot to pick k centers value brute-force

```
k1 <- kmeans(x, centers=1)
k2 <- kmeans(x, centers=2)
k3 <- kmeans(x, centers=3)
k4 <- kmeans(x, centers=4)
k5 <- kmeans(x, centers=5)
```

```
z <- c(k1$tot.withinss,
      k2$tot.withinss,
      k3$tot.withinss,
      k4$tot.withinss)
plot(z, typ="b")
```



```
n <- NULL
for(i in 1:5){
  n <- c(n, kmeans(x, centers=i)$tot.withinss)
}
plot(n, type="b")
```



Now for hclust() - Hierarchical Clustering

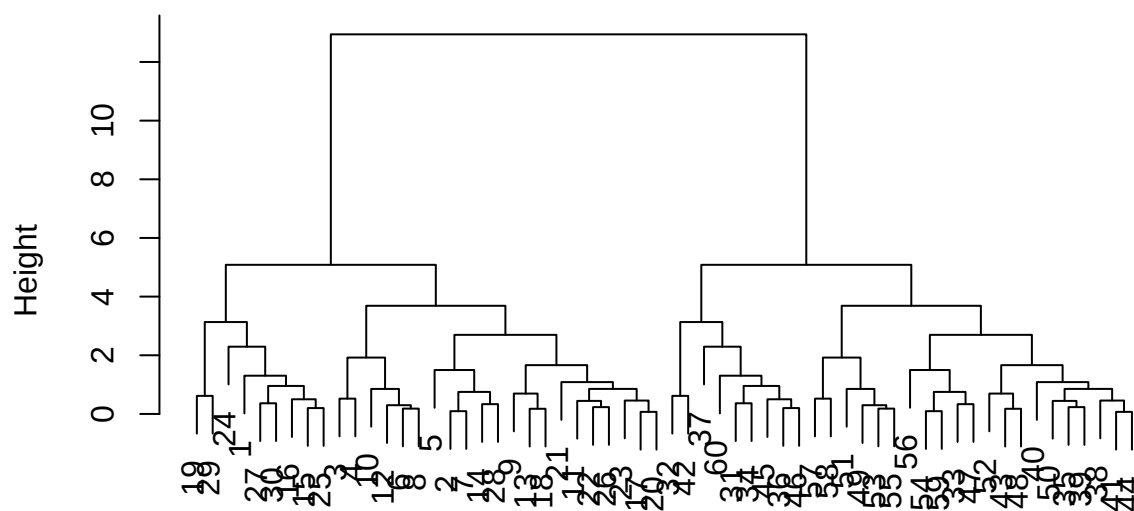
We will cluster the same data 'x' with the 'hclust()'. In this case 'hclust()' requires a distance matrix as input.

```
hclust <- hclust(dist(x))  
hclust
```

```
##  
## Call:  
## hclust(d = dist(x))  
##  
## Cluster method   : complete  
## Distance         : euclidean  
## Number of objects: 60
```

```
plot(hclust)
```

Cluster Dendrogram



```
dist(x)
hclust (*, "complete")
```

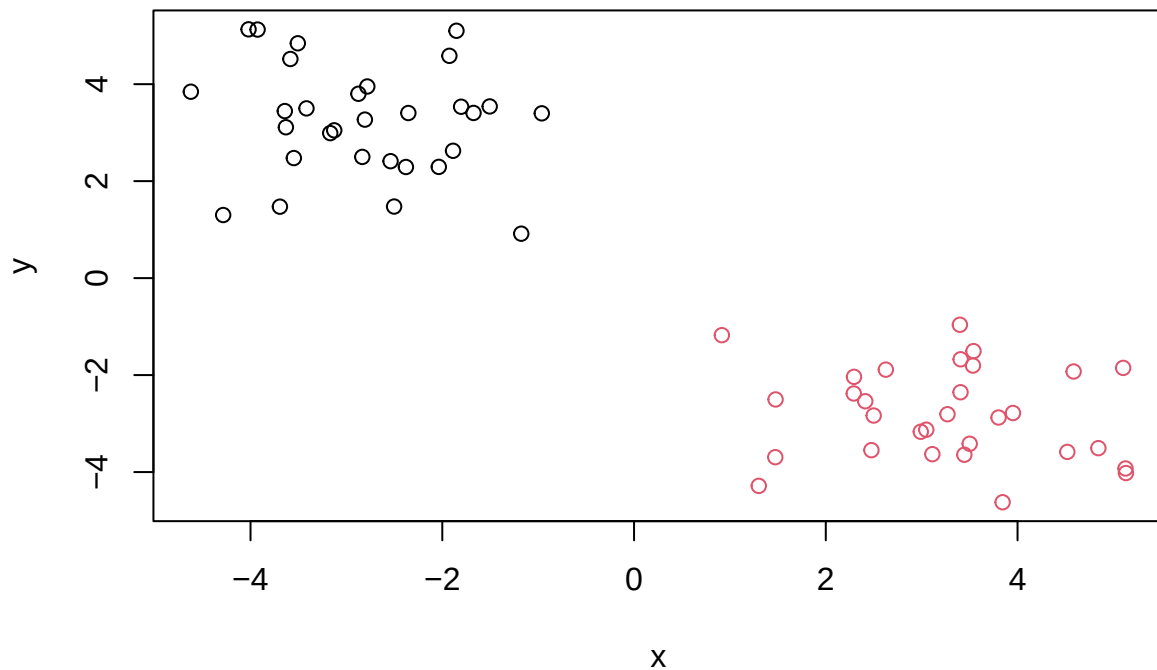
To get our cluster membership vector we need to “cut” the tree with the ‘cutree()’

```
grps <- cutree(hclust, h = 8)
grps
```

```
## [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
## [39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

Now plot the data with the hclust() results.

```
plot(x, col=grps)
```

Principal Component Analysis (PCA)

PCA of UK food data

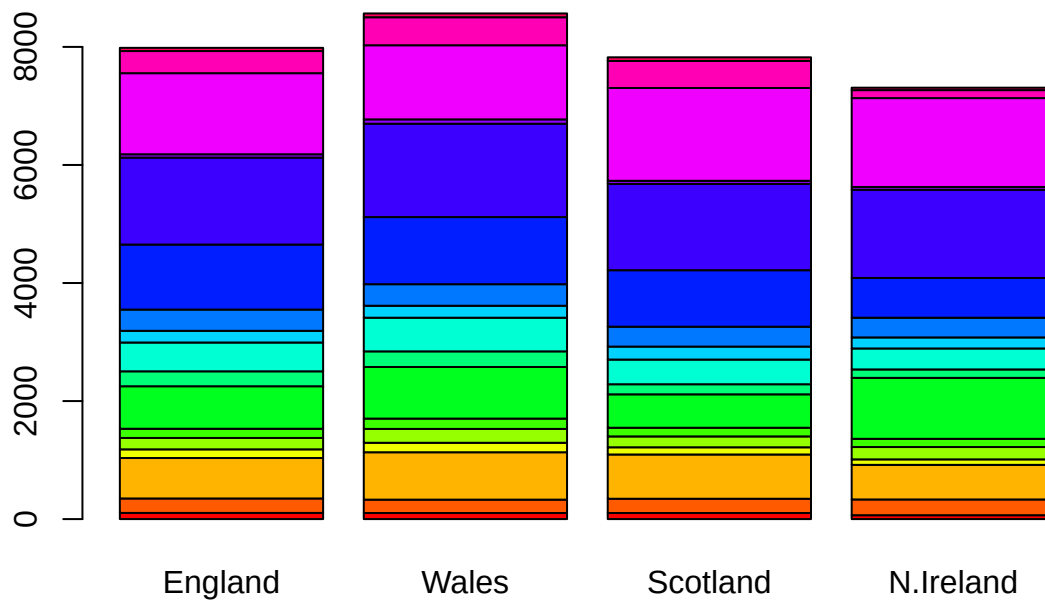
Read data from website and try a few visualizations.

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url,row.names=1)
x
```

##	England	Wales	Scotland	N.Ireland
## Cheese	105	103	103	66
## Carcass_meat	245	227	242	267
## Other_meat	685	803	750	586
## Fish	147	160	122	93
## Fats_and_oils	193	235	184	209
## Sugars	156	175	147	139
## Fresh_potatoes	720	874	566	1033
## Fresh_Veg	253	265	171	143
## Other_Veg	488	570	418	355
## Processed_potatoes	198	203	220	187
## Processed_Veg	360	365	337	334
## Fresh_fruit	1102	1137	957	674
## Cereals	1472	1582	1462	1494
## Beverages	57	73	53	47
## Soft_drinks	1374	1256	1572	1506

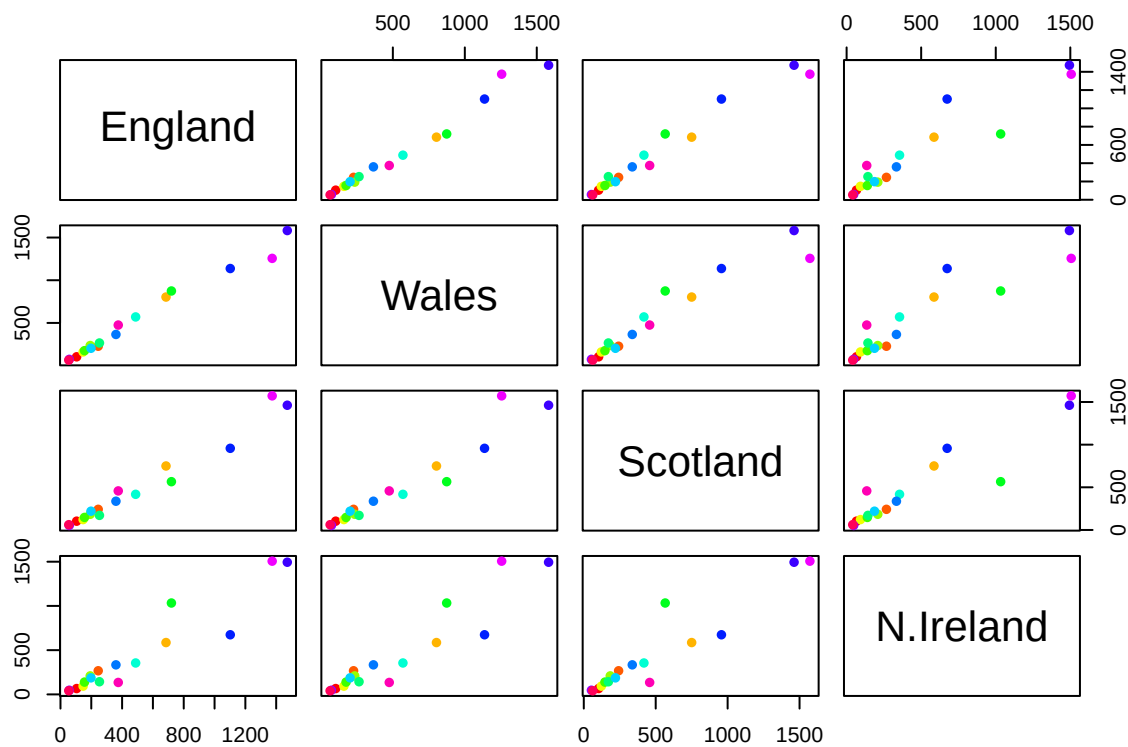
```
## Alcoholic_drinks      375   475   458   135
## Confectionery         54    64    62    41

cols <- rainbow(nrow(x))
barplot(as.matrix(x), col=cols)
```



There is one plot that can be useful for small datasets:

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



PCA to the rescue

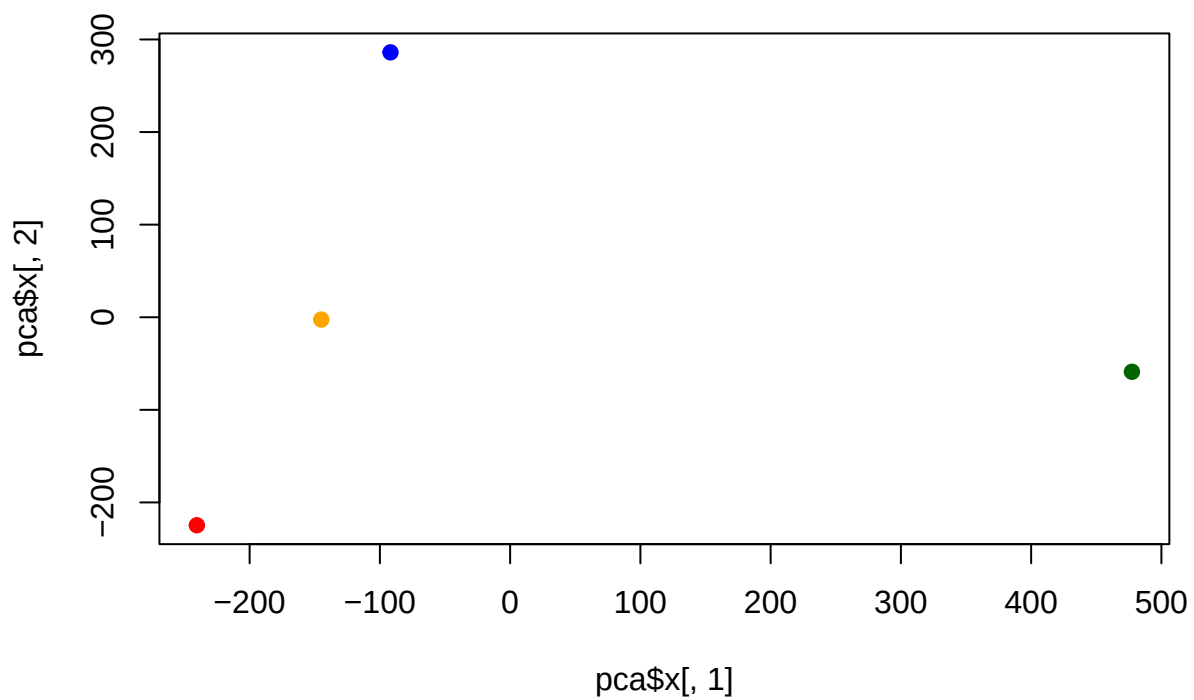
The main function for performing PCA in base R is `prcomp()`. Here, I will transpose the data so that the “foods” are arranged in the columns.

```
pca <- prcomp(t(x))
summary(pca)
```

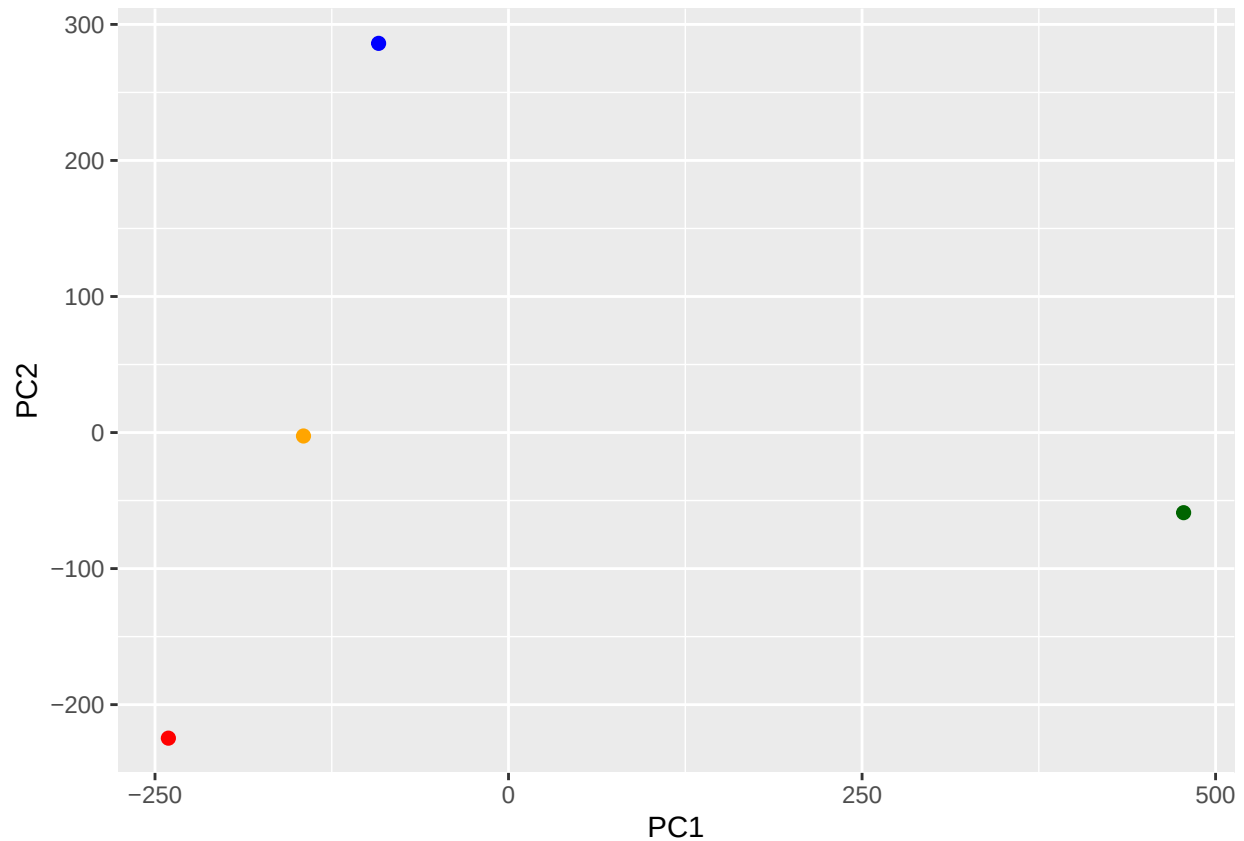
```
## Importance of components:
```

```
##              PC1      PC2      PC3      PC4
## Standard deviation 324.1502 212.7478 73.87622 3.176e-14
## Proportion of Variance 0.6744 0.2905 0.03503 0.000e+00
## Cumulative Proportion 0.6744 0.9650 1.00000 1.000e+00
```

```
cols<- c("orange", "red", "blue", "darkgreen")
plot(pca$x[,1],pca$x[,2], col=cols, pch=19)
```



```
library(ggplot2)
ggplot(pca$x) +
aes(PC1, PC2) +
geom_point(col=cols, size=2)
```



```
pca$rotation
```

	PC1	PC2	PC3	PC4
## Cheese	-0.056955380	0.016012850	0.02394295	-0.694538519
## Carcass_meat	0.047927628	0.013915823	0.06367111	0.489884628
## Other_meat	-0.258916658	-0.015331138	-0.55384854	0.279023718
## Fish	-0.084414983	-0.050754947	0.03906481	-0.008483145
## Fats_and_oils	-0.005193623	-0.095388656	-0.12522257	0.076097502
## Sugars	-0.037620983	-0.043021699	-0.03605745	0.034101334
## Fresh_potatoes	0.401402060	-0.715017078	-0.20668248	-0.090972715
## Fresh_Veg	-0.151849942	-0.144900268	0.21382237	-0.039901917
## Other_Veg	-0.243593729	-0.225450923	-0.05332841	0.016719075
## Processed_potatoes	-0.026886233	0.042850761	-0.07364902	0.030125166
## Processed_Veg	-0.036488269	-0.045451802	0.05289191	-0.013969507
## Fresh_fruit	-0.632640898	-0.177740743	0.40012865	0.184072217
## Cereals	-0.047702858	-0.212599678	-0.35884921	0.191926714
## Beverages	-0.026187756	-0.030560542	-0.04135860	0.004831876
## Soft_drinks	0.232244140	0.555124311	-0.16942648	0.103508492
## Alcoholic_drinks	-0.463968168	0.113536523	-0.49858320	-0.316290619
## Confectionery	-0.029650201	0.005949921	-0.05232164	0.001847469

PCA looks spuer useful.